# p1227R1
# Signed ssize() functions, unsigned size() functions

Published Proposal, 2019-01-21

**This version:**
http://wg21.link/p1227r1

**Author:**
Jorg Brown (Google)

**Audience:**
LEWG

**Project:**
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

---

## Abstract

This paper proposes a compromise to allow accessing container sizes as signed values, while keeping the historical precedent that size() returns a value of type size_t.

## Table of Contents

# § 1. Change history

The original version covered only the changes to std::ssize(), and the introduction of ssize() to all STL containeras. [P1227R0]

This is the second version, incorporating much of P1089R2. [P1227R1]

# § 2. Signed ssize() functions: std::ssize() for all!

## § 2.1. Introduction

When span was adopted into C++17, it used a signed integer both as an index and a size. Partly this was to allow for the use of "-1" as a sentinel value to indicate a type whose size was not known at compile time. But having an STL container whose size() function returned a signed value was problematic, so P1089 was introduced to "fix" the problem. It received majority support, but not the 2-to-1 margin needed for consensus.

This paper, P1227, was a proposal to add non-member std::ssize and member ssize() functions. The inclusion of these would make certain code much more straightforward and allow for the avoidance of unwanted unsigned-ness in size computations. The idea was that the resistance to P1089 would decrease if ssize() were made available for all containers, both through std::ssize() and as member functions.

P1089 and P1227 were discussed at length during an evening session in San Diego 2018. The next day, a new poll was taken during a special joint session between EWG and LEWG. "Proposal 4" received the highest amount of approval, however since it was not an actual paper, and had no actual author, it’s not entirely clear what it was. This paper attempts to document what I believe "Proposal 4" was.

Motivation: I believe that the objection to "P1089+P1227" was that P1227 had a clause stating that an ssize() member function should be added to all STL containers; this implied by extension that all containers should have ssize() member functions, and the amount of work to do that was strongly resisted by some in attendance. I do not believe that any part of P1089 caused objection.

In a nutshell: Every part of span that used signed values to represent a size or an index will be converted to use unsigned, except for "difference_type". Span's signed size() function will be renamed ssize() in order to adopt a size() function that returns a value of type size_t, like every other STL container. std::ssize(container) is a new function whose return value is the same as static_cast<ptrdiff_t>(std::size(container)).

## § 2.2. Motivation and scope

For motivation to use the unsigned size_t type, see http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1089r2.pdf

For motivation to use a signed size type, consider the following code:

```
template <typename T>
bool has_repeated_values(const T& container) {
  for (int i = 0; i < container.size() - 1; ++i) {
    if (container[i] == container[i + 1]) return true;
  }
  return false;
}
```

An experienced C++ programmer would immediately see the problem here, but programmers new to the language often make the mistake seen here: subtraction of 1 from a size of zero does not produce a negative value, but rather, produces a very very large positive value. So when this routine is called on an empty container, undefined behavior results.

This has been discussed ad infinitum, most recently at the standards level at the Rapperswil 2018 LEWG meeting, during a discussion of spans and ranges; the suggestion was made to put forth a proposal to explicitly add ssize() member functions to all STL containers, and to add a non-member std::ssize() function. Once these exist, programmers can simply write:

```
template <typename T>
bool has_repeated_values(const T& container) {
  for (ptrdiff_t i = 0; i < container.ssize() - 1; ++i) {
    if (container[i] == container[i + 1]) return true;
  }
  return false;
}
```

## § 3. Impact on the Standard

This proposal is a pure library extension that could be implemented in C++11.

## § 4. Proposed Wording

Modify 21.7.2 [span.syn] :

```
inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();
template <class ElementType, ptrdiff_t Extent = dynamic_extent>
class span;
template <class T, size_t X, class U, size_t Y>
constexpr bool operator==(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator!=(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator<(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator<=(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator>(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>

constexpr bool operator>=(span<T, X> l, span<U, Y> r);
template <class ElementType, size_t Extent>
span<const byte,
Extent == dynamic_extent ? dynamic_extent
: sizeof(ElementType) * Extent>
as_bytes(span<ElementType, Extent> s) noexcept;
template <class ElementType, ptrdiff_t Extent>
span<byte,
Extent == dynamic_extent ? dynamic_extent
: sizeof(ElementType) * Extent>
as_writable_bytes(span<ElementType, Extent> s) noexcept;
```

Change span synopsis [span.overview]

~~3 If Extent is negative and not equal to dynamic_extent, the program is ill-formed.~~

```
template <class ElementType, size_t Extent = dynamic_extent>
class span {
using index_type = size_t;
template <class OtherElementType, size_t OtherExtent>
constexpr span(const span<OtherElementType, OtherExtent>& s)
noexcept;
template <size_t Count>
constexpr span<element_type, Count> first() const;
template <size_t Count>
constexpr span<element_type, Count> last() const;
template <size_t Offset, size_t Count =
dynamic_extent>
constexpr span<element_type, see below > subspan() const;
```

Change [span.sub]

```
template <size_t Count> constexpr span<element_type, Count>
first() const;
1. Requires: Count <= size().

template <size_t Count> constexpr span<element_type, Count>
last() const;

3. Requires: Count <= size().

template <size_t Offset, size_t Count = dynamic_extent>
constexpr span<element_type, see below > subspan() const;

5. Requires:
(Offset <= size()) && (Count == dynamic_extent || Offset + Count <= size())

8. Requires: count <= size().

10. Requires: count <= size().

12. Requires: (offset <= size())
&& (count == dynamic_extent || offset + count <= size())
```

Change [spam.elem]

```
1. Requires: idx < size().
```

Change [span.comparison]

```
template <class T, size_t X, class U, size_t Y>
constexpr bool operator==(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator!=(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator<(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator<=(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator>(span<T, X> l, span<U, Y> r);
template <class T, size_t X, class U, size_t Y>
constexpr bool operator>=(span<T, X> l, span<U, Y> r);
```

Change [span.objectrep]

```
template <class ElementType, size_t Extent>
span<const byte, Extent == dynamic_extent ? dynamic_extent
: sizeof(ElementType) * Extent>
as_bytes(span<ElementType, Extent> s) noexcept;

template <class ElementType, ptrdiff_t Extent>
span<byte, Extent == dynamic_extent ? dynamic_extent
: sizeof(ElementType) * Extent>
as_writable_bytes(span<ElementType, Extent> s) noexcept;
```

Modify the section 27.3 Header synopsis [iterator.synopsis] by adding the following near the declarations for size():

```
  template <class C>
  constexpr ptrdiff_t ssize(const C& c);

  template <class T, ptrdiff_t N>
  constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
```

Modify the section 27.8 Container access by adding the following near the definitions for size():

```
template <class C>
constexpr ptrdiff_t ssize(const C& c);
  // Returns: static_cast<ptrdiff_t>(c.size())

template <class T, ptrdiff_t N>
constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
  // Returns: N.
```

Note that the code does not just return the std::make_signed variant of the container's size() method, because it's conveivable that a container might choose to represent its size as a uint16_t, supporting up to 65,535 elements, and it would be a disaster for std::ssize() to turn a size of 60,000 into a size of -5,536.

## § 5. Acknowledgements

Thanks to Riccardo Marcangelo, whose [N4017] proposal contains verbiage too good not to copy.

And of course, thanks to Robert Douglas, Nevin Liber, and Marshall Clow, whose P1089R2 I directly copied from, in order to make this document.

## § References

### § Informative References

**[N4017]**
Riccardo Marcangelo. Non-member size() and more. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4017.htm

**[P1227R0]**
Jorg Brown. Signed size() functions. URL: http://wg21.link/p1227r0

**[P1227R1]**
Jorg Brown. Signed ssize() functions, unsigned size() functions. URL: http://wg21.link/p1227r1